SlipGate — Complete Operator Guide

> The official server for the SlipNet Android app. Source: https://github.com/anonvector/slipgate SlipNet channel: @SlipNet_app

---

Table of Contents

1. What SlipGate is
2. What you need before starting
3. Picking a VPS
4. Setting up your domain & DNS records
5. Installing SlipGate
6. The interactive menu vs. CLI
7. Adding your first tunnel
8. Tunnel transports — when to use which
9. Adding & managing users
10. Sharing configs with clients (`slipnet://` URIs)
11. Running multiple tunnels at once
12. WARP outbound (optional)
13. Daily operations: stats, logs, diagnostics
14. Updating SlipGate
15. Backup & migration
16. A note on VLESS
17. Troubleshooting
18. File locations & systemd reference
19. Uninstalling

---

1. What SlipGate is

SlipGate is a one-binary tunnel manager for Linux servers. You run it on your VPS, and it sets up the server side of every protocol SlipNet supports — DNSTT, NoizDNS, VayDNS, Slipstream, NaiveProxy, and StunTLS — with `systemd` services, a DNS router, user management, a live dashboard, and `slipnet://` URI generation for one-tap import in the SlipNet app.

In practice: **one command** turns a fresh Ubuntu/Debian VPS into a full anti-censorship endpoint that your phone (and your friends' phones) can connect to.

What SlipGate does for you:

- Installs the right transport binaries (`dnstt-server` , `slipstream-server` , `vaydns-server` , `caddy-naive`).

- Generates Curve25519 keys per tunnel.
- Configures `systemd` units that auto-start on boot.
- Runs a built-in SOCKS5 proxy and a DNS router that dispatches queries by domain.
- Manages users (one credential, all tunnels).
- Auto-generates Let's Encrypt certs for NaiveProxy via Caddy.
- Self-updates from GitHub releases.

---

2. What you need before starting

- A **Linux VPS** with **root SSH access** — Ubuntu 20.04+, Debian 10+, or any modern systemd-based distro.
- A **domain name** with control over its DNS records (Cloudflare, Namecheap, Porkbun, your registrar's panel, anything works).
- **Open ports** on the server:
  - `53/udp` for DNS tunnels (DNSTT, NoizDNS, VayDNS, Slipstream)
  - `443/tcp` for NaiveProxy and StunTLS
- About **15 minutes** of your time.
- **Cost estimate:** a $4–6/month VPS handles personal use comfortably.

---

3. Picking a VPS

The "right" VPS depends on where your users are.

Avoid providers that block UDP/53 inbound, since that kills DNS tunnels:

- DigitalOcean blocks UDP/53 on most plans.
- AWS blocks UDP/53 on Lightsail and on EC2 with default security groups.
- Some big US clouds throttle UDP heavily.

Good choices for serving users in Iran (and similar):

- **Hetzner** (Germany / Finland) — cheap, fast, doesn't block ports.
- **Contabo** (Germany) — very cheap, generous bandwidth.
- **OVH** (Europe) — no port restrictions.
- **BuyVM** — small but robust.
- Smaller European or Asian providers — usually fine.

Pick a region that's geographically close to your users but not blocked by your ISP. For Iran users, Germany / Finland / Netherlands are typical sweet spots.

Sizing: 1 vCPU, 1 GB RAM, 20 GB disk is enough for a personal/small-group server. Bandwidth matters more than CPU.

---

4. Setting up your domain & DNS records

You need one domain (e.g., `example.com`) that you control. Add these records before installing SlipGate:

```
A    ns.example.com   →  <server IP>
A    example.com      →  <server IP>          (only if using NaiveProxy / StunTLS)

NS   t.example.com    →  ns.example.com       (DNSTT/NoizDNS — SOCKS backend)
NS   ts.example.com   →  ns.example.com       (DNSTT/NoizDNS — SSH backend)
NS   s.example.com    →  ns.example.com       (Slipstream — SOCKS backend)
NS   ss.example.com   →  ns.example.com       (Slipstream — SSH backend)
NS   v.example.com    →  ns.example.com       (VayDNS — SOCKS backend)
NS   vs.example.com   →  ns.example.com       (VayDNS — SSH backend)
```

You only need the records for the tunnel types you plan to use. **Minimum**:

- For NaiveProxy only: just the root `A example.com → <IP>`.
- For DNSTT only: `A ns.example.com → <IP>` plus `NS t.example.com → ns.example.com`.

Important about NS delegation: the `NS` records mean "queries for `t.example.com` go to `ns.example.com`, which lives at `<server IP>`." Your server's port 53 will answer them via the DNS tunnel binary.

DNS changes propagate within a few minutes to a few hours. Verify with:

```
dig +short NS t.example.com
dig +short A ns.example.com
```

> If you're using **Cloudflare** for DNS, set the records to **DNS only** (gray cloud). Don't proxy them — the tunnel needs raw DNS at the server.

---

5. Installing SlipGate

SSH into your server as root, then run the installer:

```
curl -fsSL https://raw.githubusercontent.com/anonvector/slipgate/main/install.sh | sudo
```

The script:

- Detects your distro.
- Installs system packages it needs.
- Downloads the SlipGate binary plus `dnstt-server`, `slipstream-server`, `vaydns-server`, and `caddy-naive`.
- Installs systemd services (`slipgate-dnsrouter`, `slipgate-socks5`).
- Sets up the firewall rules where applicable.

Offline install (when your VPS can't reach GitHub directly):

1. On your laptop, download from https://github.com/anonvector/slipgate/releases.
2. SCP the binaries to the server: `scp slipgate-* user@server:/tmp/slipgate/`
3. On the server: `sudo slipgate install --bin-dir /tmp/slipgate`

After install, launch the interactive menu:

```
sudo slipgate
```

---

6. The interactive menu vs. CLI

SlipGate has two interfaces:

Interactive TUI — `sudo slipgate` with no arguments. A friendly menu that walks you through tunnels, users, stats, diagnostics, etc. Best for first-time setup and exploration.

CLI subcommands — for scripting and automation. Every interactive option has a CLI equivalent:

```
slipgate install                Install dependencies, set up server
slipgate uninstall              Remove everything
slipgate update                 Self-update + restart services
slipgate restart                Restart all services
slipgate users                  Add / list / remove users (interactive)
slipgate users add              Add one user
slipgate users bulk_add         Add many users with random passwords
slipgate users list             List users + per-tunnel configs
slipgate stats                  Live dashboard
slipgate diag                   Run diagnostic checks
slipgate mtu [value]            Set MTU on all DNS tunnels at once

slipgate tunnel add             Add a tunnel
slipgate tunnel edit [tag]      Edit settings (rename, MTU, keys)
slipgate tunnel remove [tag]    Remove one tunnel
slipgate tunnel remove --all    Remove all tunnels
slipgate tunnel start [tag]     Start a tunnel
slipgate tunnel stop [tag]      Stop a tunnel
slipgate tunnel status [tag]    Show status / details
slipgate tunnel share [tag]     Print slipnet:// URI + QR
slipgate tunnel logs [tag]      Show systemd logs

slipgate router status          DNS routing config
slipgate router mode            Switch single ↔ multi mode
slipgate router switch          Change active tunnel (single mode)

slipgate config export          Export full config (JSON)
slipgate config import          Import config back in
```

---

7. Adding your first tunnel

The simplest possible setup — a DNSTT tunnel with the built-in SOCKS5 backend:

```
sudo slipgate tunnel add \
  --transport dnstt \
  --backend socks \
  --tag mytunnel \
  --domain t.example.com
```

What this does:

1. Generates a fresh Curve25519 keypair.
2. Spins up `dnstt-server` listening on UDP/53 for `t.example.com`.
3. Forwards decrypted traffic into the built-in SOCKS5 proxy on `127.0.0.1:1080`.
4. Creates a systemd service `slipgate-mytunnel` that starts on boot.

Confirm it's running:

```
sudo slipgate tunnel status
```

You should see your tunnel listed as **active**. If anything looks off, run:

```
sudo slipgate diag
```

---

8. Tunnel transports — when to use which

SlipGate manages six transports. Pick based on the network your users have to escape from.

Transport	Port	What it is	Use when
DNSTT	53/udp	Encrypted DNS tunnel (Curve25519). Mature, reliable	Default; also serves NoizDNS clients on the same port
NoizDNS	53/udp	Same server as DNSTT — clients enable DPI evasion	Network does DNS-tunnel detection
VayDNS	53/udp	KCP-based DNS tunnel with tunable wire format	Advanced; flaky DNS paths; need per-record-type tweaks
Slipstream	53/udp	QUIC over DNS, very fast	Clean networks; users want speed
NaiveProxy	443/tcp	Caddy + forwardproxy plugin, real Let's Encrypt cert, decoy site	DNS-tunneling banned; HTTPS still works
StunTLS	443/tcp	SSH-over-TLS / SSH-over-WebSocket, self-signed cert	No domain available; or want CDN-friendly WS layer

Backends for each transport:

- `socks` — built-in Go SOCKS5 proxy. Simple, fast, no SSH overhead.
- `ssh` — server's OpenSSH does the forwarding. Zero DNS leaks, slightly more secure.
- `both` — creates two tunnels in parallel (e.g., `mytunnel-socks` and `mytunnel-ssh`). Auto-generates the second subdomain by appending `s`.

Examples

```
# VayDNS — newer DNS tunnel, optimized
sudo slipgate tunnel add --transport vaydns --backend socks \
  --tag myvaydns --domain v.example.com

# VayDNS with all tuning knobs
sudo slipgate tunnel add --transport vaydns --backend both \
  --tag myvaydns --domain v.example.com \
  --record-type txt --idle-timeout 10s --keep-alive 2s \
  --clientid-size 2 --queue-size 512

# Slipstream over QUIC
sudo slipgate tunnel add --transport slipstream --backend ssh \
  --tag myslip --domain s.example.com

# NaiveProxy — auto Let's Encrypt cert
sudo slipgate tunnel add --transport naive --backend socks \
  --tag myproxy --domain example.com \
  --email admin@example.com --decoy-url https://www.wikipedia.org

# StunTLS — SSH over TLS + WebSocket, no domain needed
sudo slipgate tunnel add --transport stuntls --tag mytls

# Direct SSH or SOCKS5 (no tunnel transport)
sudo slipgate tunnel add --transport direct-ssh --tag myssh
sudo slipgate tunnel add --transport direct-socks5 --tag mysocks

# External: forward DNS for a domain to a custom UDP port (BYO protocol)
sudo slipgate tunnel add --transport external --tag my-proto \
  --domain j.example.com --port 5301
```

---

9. Adding & managing users

Users in SlipGate are **global**. The same `username:password` authenticates against every tunnel, regardless of transport. The protocol is a property of the **tunnel**, not the user.

> ⚠️ **Never hand out the server's `root` (or any sudoer / shell login) account as a VPN credential.** SlipGate users are isolated SOCKS/SSH-forwarding accounts with no shell access — losing one only loses tunnel access. A leaked `root` account loses the whole server: your keys, your other users' credentials, and any other service running on the box. Always create dedicated SlipGate users with `slipgate users add` (or `bulk_add`) for everyone you give a config to, including yourself.

Add one user (interactive):

```
sudo slipgate users add
```

You'll be prompted for username and password.

Bulk-create users (random passwords, up to 500 in one go):

```
sudo slipgate users bulk_add --count=50 --prefix=user
# Creates user001..user050
```

List all users with their per-tunnel configs:

```
sudo slipgate users list
```

This prints one config block per (user × tunnel) pair, including the `slipnet://` URI for each.

Remove a user:

```
sudo slipgate users remove
```

> Tip: if you're running a small group, give each person a unique username so you can revoke just theirs later if a phone is lost.

---

10. Sharing configs with clients

After creating a tunnel, generate a `slipnet://` URI for one-tap import in the SlipNet app:

```
sudo slipgate tunnel share mytunnel
```

This prints:

- A `slipnet://...` URI (long base64 string).
- A QR code (in the TUI).

Send the URI to your user. They open SlipNet → **+** → **Import from URI** → paste → done.

For DNSTT tunnels, the share command asks whether to generate a **DNSTT** or **NoizDNS** profile. The same server handles both — NoizDNS just enables DPI evasion on the client side, so:

- **DNSTT** profile: a bit faster, easier to detect.
- **NoizDNS** profile: slower, harder to fingerprint.

> ⚠️ The URI contains the user's credentials. Send it via a private channel, never in a public group.

---

11. Running multiple tunnels at once

You can run as many tunnels as you want. SlipGate auto-switches into **multi-mode** when you create a second DNS tunnel — the DNS router on port 53 dispatches queries by domain to the correct tunnel.

Example: DNSTT + VayDNS + NaiveProxy at the same time:

```
sudo slipgate tunnel add --transport dnstt   --backend socks --tag dnstt1   --domain t.
sudo slipgate tunnel add --transport vaydns  --backend socks --tag vaydns1  --domain v.
sudo slipgate tunnel add --transport naive   --backend socks --tag naive1   --domain ex
```

Each tunnel becomes its own systemd service (`slipgate-dnstt1` , etc.). Manage them like any service:

```
sudo slipgate tunnel status
sudo slipgate tunnel start vaydns1
sudo slipgate tunnel stop vaydns1
sudo slipgate tunnel logs naive1
```

Routing modes

- **Single mode** — one DNS tunnel active; port 53 forwards to it.
- **Multi mode** — all DNS tunnels run on local ports; port 53 dispatches by domain.

```
sudo slipgate router status      # see current mode
sudo slipgate router mode        # switch between single and multi
sudo slipgate router switch      # change active tunnel (single mode only)
```

---

12. WARP outbound (optional)

Cloudflare WARP can be enabled as the server's **outbound** path — useful when:

- Your VPS provider's IP is blacklisted by sites your users want to reach (Netflix, OpenAI, ChatGPT, banking).
- You want to add an extra layer between the tunnel server and the public internet.

Enable it from the interactive menu (**WARP** option). SlipGate sets up a `wireguard-go` link to WARP and routes user traffic through it.

> Trade-off: a small latency penalty, and you depend on Cloudflare. Skip it if your VPS IP already reaches everywhere your users need.

---

13. Daily operations

Live dashboard — CPU, RAM, traffic sparklines, per-protocol connection counts, tunnel status:

```
sudo slipgate stats
```

Diagnostics — checks services, ports, keys, DNS resolution, boot persistence:

```
sudo slipgate diag
```

Logs for a specific tunnel:

```
sudo slipgate tunnel logs mytunnel
```

Restart everything (DNS router, SOCKS, all tunnels):

```
sudo slipgate restart
```

Tune MTU on all DNS tunnels at once — useful if links are flaky:

```
sudo slipgate mtu 1200
# Common values: 1232 (default), 1200 (cautious), 1100 (very flaky links)
```

Tweak one tunnel:

```
sudo slipgate tunnel edit --tag mydnstt --new-tag my-tunnel  # rename
sudo slipgate tunnel edit --tag mydnstt --mtu 1232           # change MTU
sudo slipgate tunnel status --tag mydnstt                    # keys, port, MTU
```

---

14. Updating SlipGate

Pulls the latest binary from GitHub releases and restarts services:

```
sudo slipgate update
```

The update is in-place — your tunnels, users, and keys are untouched. Existing client `slipnet://` URIs keep working.

---

15. Backup & migration

Export your full config (tunnels, users, keys, certs):

```
sudo slipgate config export > slipgate-backup.json
```

Save this somewhere safe (your laptop, encrypted backup).

Restore on a new server:

```
sudo slipgate install
sudo slipgate config import < slipgate-backup.json
```

This is also how you migrate to a new VPS without breaking client URIs:

1. Spin up a new VPS.
2. Update your domain's `A` record to the new IP.
3. Install SlipGate on the new VPS and import the backup.
4. Existing `slipnet://` URIs keep working — same keys, same domain.

---

16. A note on VLESS

The SlipNet client supports **VLESS** (over WebSocket / TLS, typically fronted through Cloudflare). **SlipGate does not run a VLESS server itself** — VLESS is part of the Xray / V2Ray ecosystem.

If you want to give your users a VLESS option, you have two paths:

1. **Run Xray separately on the same VPS.** Standard Xray installers (e.g., `Xray-install`, `x-ui`, `3x-ui`) work fine alongside SlipGate, since they typically listen on a different port (or on 443 if SlipGate isn't using NaiveProxy/StunTLS). Your users build a manual VLESS profile in SlipNet using the UUID, domain, and CDN settings from your Xray panel.
2. **Skip VLESS** — SlipGate's six transports already cover most censorship environments. NaiveProxy and StunTLS solve the "443 is the only port left" problem.

VLESS support may land in SlipGate in the future. Track the channel for updates.

---

17. Troubleshooting

Symptom	Try this
Tunnel won't start	`sudo slipgate diag` — checks ports, keys, DNS records
Client connects but no traffic	`sudo slipgate tunnel logs <tag>` for handshake / auth errors
Port 53 already in use	`sudo systemctl disable --now systemd-resolved` (then add `nameserver 1.1.1.1` to `/etc/resolv.conf`)
Let's Encrypt fails (NaiveProxy)	A record may not have propagated, or port 80 is blocked. Wait, then `sudo slipgate restart`
`dig` against your server times out	VPS provider blocks UDP/53 inbound. Switch provider or stick to NaiveProxy/StunTLS on 443
User can't authenticate	`sudo slipgate users list` — confirm credentials present; check tunnel has matching backend
Slow DNS tunnel	Lower MTU: `sudo slipgate mtu 1200`
Multiple tunnels but only one works	`sudo slipgate router status` — confirm multi-mode is active
Need to reset a key	`sudo slipgate tunnel edit --tag <tag>` and regenerate; clients will need a fresh URI

For deeper inspection, every tunnel is a regular systemd service:

```
journalctl -u slipgate-mytunnel -n 200 --no-pager
journalctl -u slipgate-dnsrouter -n 200 --no-pager
journalctl -u slipgate-socks5 -n 200 --no-pager
```

---

18. File locations & systemd reference

Path	Description
`/etc/slipgate/config.json`	Main configuration (tunnels, users, routing mode)
`/etc/slipgate/tunnels/`	Per-tunnel keys, certs, configs
`/usr/local/bin/slipgate`	SlipGate binary
`/usr/local/bin/dnstt-server`	DNSTT/NoizDNS transport
`/usr/local/bin/slipstream-server`	Slipstream transport
`/usr/local/bin/vaydns-server`	VayDNS transport
`/usr/local/bin/caddy-naive`	NaiveProxy (Caddy + forwardproxy plugin)
`/etc/caddy/Caddyfile`	Caddy config (managed by SlipGate)

systemd units:

- `slipgate-dnsrouter` — port 53 DNS dispatcher
- `slipgate-socks5` — built-in SOCKS5 proxy
- `slipgate-<tag>` — one per tunnel you create

---

19. Uninstalling

```
sudo slipgate uninstall
```

Removes all services, configs, and binaries. Your DNS records are untouched — delete them manually in your DNS provider's panel if you want.

Channel: @SlipNet_app SlipGate source: https://github.com/anonvector/slipgate SlipNet source: https://github.com/anonvector/SlipNet